

Two-dimensional arrays

CSC103 Programming Fundamentals / Lecture 22

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Compute the total of every element in $\{\{2,4\},\{6,8\}\}$.
Then compute its main diagonal sum.

Traverse every element with nested loops to accumulate the grand total.

Learning objectives

- Create and access a table of values
- Traverse rows and columns
- Compute row and column totals

CLO-2

Arrays of arrays

Nested traversal

Row totals

Column totals

Arrays of arrays

- A Java two-dimensional array is an array of row references.
- `data[r][c]` selects column `c` in row `r`.
- A rectangular example has equal row lengths.

Example

```
int[] [] marks = {  
    {60, 70, 80},  
    {50, 65, 75}  
};
```

Nested traversal

- Use the outer length for row count.
- Use each row length for its column count.
- Keep the row and column indexes distinct.

Example

```
for (int r=0; r<data.length; r++) {  
    for (int c=0; c<data[r].length; c++) {  
        System.out.println(data[r][c]);  
    }  
}
```

Row totals

- Reset a row accumulator at the start of each row.
- The inner loop adds elements from that row.
- A row mean divides by the row length only when nonempty.

Example

```
for (int[] row : data) {  
    int total = 0;  
    for (int value : row) total += value;  
    System.out.println(total);  
}
```

Column totals

- Column traversal reverses the roles of the indexes.
- A rectangular-matrix contract simplifies column totals.
- Check shape assumptions before using `data[0]`.

Example

For $\{\{1,2\},\{3,4\}\}$:

Column 0 total: $1 + 3 = 4$

Column 1 total: $2 + 4 = 6$

Different traversals answer different questions

- A row traversal keeps the row fixed while the column changes.
- A column traversal keeps the column fixed while the row changes.
- A main-diagonal traversal selects entries whose row and column indexes match.

Example

For $\{\{2,4\},\{6,8\}\}$:

Row 0: 2,4

Column 0: 2,6

Main diagonal: 2,8

Accumulator placement changes the meaning

- A grand total starts before both loops.
- A row total resets inside the outer loop.
- A diagonal sum needs one selected element per suitable row.

Example

Grand total: $2 + 4 + 6 + 8 = 20$

Diagonal total: $2 + 8 = 10$

Rows and columns identify one element

	column 0	column 1	column 2
row 0	2	4	6
row 1	1	3	5

What to notice

For this 2 by 3 array, `data[1][2]` is 5: row 1, column 2.

Key distinctions

Row / column	Column 0	Column 1	Row total
Row 0	2	4	6
Row 1	6	8	14
Column total	8	12	20 overall

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int[][] data = {{1, 2, 3}, {4, 5, 6}};  
for (int[] row : data) {  
    int total = 0;  
    for (int value : row) total += value;  
    System.out.println(total);  
}
```

First example: result

6

15

A fresh total starts at zero for each row.

Row totals

Column totals

Guided practice

Compute the total of every element in $\{\{2,4\},\{6,8\}\}$.
Then compute its main diagonal sum.

Attempt the solution before
continuing.
Use the definitions and examples
from this lecture.

Solution strategy

- Traverse every element with nested loops to accumulate the grand total.
- For this square matrix, add `data[i][i]` in a separate loop.
- Report each result with a distinct label.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution22 {  
    public static void main(String[] args) throws Exception {  
        int[][] data = {{2, 4}, {6, 8}};  
        int total = 0;  
        for (int[] row : data) {  
            for (int value : row) total += value;  
        }  
        int diagonal = 0;  
    }  
}
```


Complete solution (2/2)

```
    for (int i = 0; i < data.length; i++) {  
        diagonal += data[i][i];  
    }  
    System.out.println("Total: " + total);  
    System.out.println("Diagonal: " + diagonal);  
}  
}
```

Execution trace

Selected element	Value	Running grand total
data[0][0]	2	2
data[0][1]	4	6
data[1][0]	6	12
data[1][1]	8	20

Follow the changing state in execution order.

Solution result

Total: 20
Diagonal: 10

Why this result follows
Report each result with a distinct
label.

A common incorrect approach

```
for(int c=0; c<data.length; c++) {  
    total += data[r][c];  
}
```

Example

The bound uses the number of rows. Use `data[r].length` for columns in row `r`.

Boundary and failure checks

Check the stated input assumptions.
Represent marks for three students in two quizzes. Print each student total and each quiz average.

All-element sum: 20. Main diagonal:
 $\text{data}[0][0] + \text{data}[1][1] = 10$. For
general diagonal traversal use the
declared matrix shape contract.

Check your understanding

What does `data.length` measure? Where should a row-total accumulator be reset?

Write a prediction and explain the rule that supports it.

Answer and explanation

The number of rows in the outer array. Reset it inside the outer row loop before the inner traversal.

The bound uses the number of rows. Use `data[r].length` for columns in row `r`.

Compare cases and predict outcomes

Aggregate	Elements	Result
Row 0	$2 + 4 + 6$	12
Column 1	$4 + 3$	7
All elements	$2 + 4 + 6 + 1 + 3 + 5$	21

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

For $\{\{2,4,6\},\{1,3,5\}\}$, compute column 1 using a loop over rows. State the shape assumption.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int[] [] data = {{2, 4, 6}, {1, 3, 5}};  
int total = 0;  
for (int row = 0; row < data.length; row++) {  
    total += data[row][1];  
}  
System.out.println(total);
```

- Column 1 contains 4 and 3, so the output is 7.
- Every visited row must be non-null and contain index 1.
- For arbitrary ragged data, check each row before accessing that column.

Grading a multiple-choice test

- Rows represent students and columns represent questions.
- Compare each answer with the key at the same column index.
- Reset the score for each student, so scores do not accumulate across rows.

Source: [supplied ISB campus slides](#). See [speaker notes and ISB_Source_Map.md](#).

Worked problem: Grading a multiple-choice test

Task

Use the first three source students and answer key $\{D,B,D,C,C\}$. Compute each score out of five.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/2)

```
public class IsbLecture22 {  
    public static void main(String[] args) throws Exception {  
        char[][] answers = {  
            {'A','B','A','C','C'},  
            {'D','B','A','B','C'},  
            {'E','D','D','A','C'}  
        };  
        char[] key = {'D','B','D','C','C'};  
        for (int student = 0; student < answers.length; student++) {
```

Complete ISB example (2/2)

```
int score = 0;
for (int question = 0; question < key.length; question++) {
    if (answers[student][question] == key[question]) score++;
}
System.out.println("Student " + (student + 1) + ": " + score);
}
}
```

Worked trace and expected result

Student	Correct question numbers	Score
1	2, 4, 5	3
2	1, 2, 5	3
3	3, 5	2

Expected console output

```
Student 1: 3  
Student 2: 3  
Student 3: 2
```

Extend the ISB example

Practice

Which assumption must be checked when rows come from an input file?

Explain your reasoning

Every student row must have the same number of answers as the key; reject malformed row lengths.

Lecture summary

- and access a table of values
 - Traverse rows and columns
 - row and column totals
- Traverse every element with nested loops to accumulate the grand total.
 - For this square matrix, add `data[i][i]` in a separate loop.
 - Report each result with a distinct label.

After-class practice

Represent marks for three students in two quizzes. Print each student total and each quiz average.

Syllabus reading

Deitel Ch 7

Next lecture: Ragged and multidimensional arrays